Introduction to Python

Session 14

Third Party Packages

Data Science:

numpy
pandas
matplotlib
scipy
sklearn

NumPy

`numpy`

What **`numpy`** offers:

- Powerful N-dimensional **array** object.
- Tools for integrating C/C++ and Fortran codes
- Linear algebra, Fourier Transform, Random numbers
- etc.

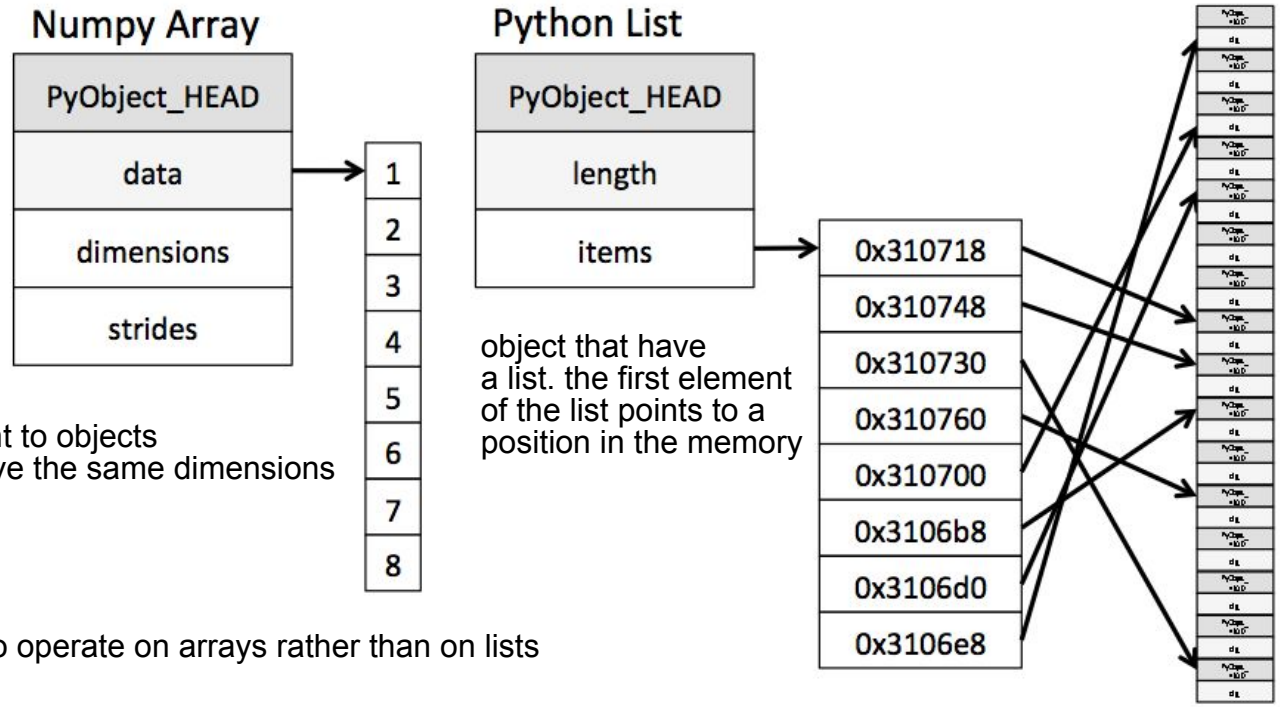
ndarray: main numpy object.
Homogeneous multidimensional array.

```
>>> import numpy as np

>>> a = np.array( [1,2,3,4] )

>>> a
array([1, 2, 3, 4])

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )
>>> b
array([[1, 2],
       [3, 4],
       [5, 6]])
```

in this case is an array of three dimentions

have mutlidimentional arrays

ndarray: main numpy object.
Homogeneous multidimensional array.



object that have
a list. the first element
of the list points to a
position in the memory

they do not point to objects
all elements have the same dimensions

better to operate on arrays rather than on lists

basic operations we could do on an array

ndarray: main numpy object.

Attributes: **size**

```
>>> import numpy as np

>>> a = np.array( [1,2,3,4] )

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> a.size
4
>>> b.size
6
```

ndarray: main numpy object.

Attributes: **ndim**

```
>>> import numpy as np

>>> a = np.array( [1,2,3,4] )

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> a.ndim
1
>>> b.ndim
2
```

because it has lists in a list

ndarray: main numpy object.

Attributes: **shape**

```
>>> import numpy as np

>>> a = np.array( [1,2,3,4] )

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> a.shape
(4,)
>>> b.shape
(3, 2)
```

ndarray: main numpy object.

Attributes: **dtype**

do not mix data types, int or str

```
>>> import numpy as np

>>> a = np.array( [1,2,3,4] )

>>> a.dtype
dtype('int64')

>>> b = np.array( [ ["a","b"], ["c","d"], ["ef","gh"] ] )
>>> b.dtype
dtype('<U2')

>>> b = np.array( [ ["a","b"], ["c","d"], ["e","g"] ] )
>>> b.dtype
dtype('<U1')
```

Numpy has its own data types. The conversion is done automatically.

ndarray: main numpy object.

Attributes: **dtype**

```
>>> import numpy as np

>>> a = np.array( [ [1,2], ["a",3] ] )
>>> a
array([['1', '2'],
       ['a', '3']],
      dtype='<U21')
```

Numpy arrays are homogeneous! They cannot contain elements of different types

ndarray: main numpy object.

Indexing and slicing

```
>>> import numpy as np

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> b[1]
array([3, 4])

>>> b[0:2]
array([[1, 2],
       [3, 4]])

>>> b[1,1]
4
```

ndarray: main numpy object.

Indexing and slicing

```
>>> import numpy as np

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> b[:,1]
array( [2,4,6] )

>>> b[0:2, 0]
array( [1,3] )

>>> b[0:2,]
array( [[1, 2],
        [3, 4]] )
```

ndarray: main numpy object.

Iterating

```
>>> import numpy as np

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> for row in b:
...     print(row)
...
[1 2]
[3 4]
[5 6]
```

iterate through a nested list

ndarray: main numpy object.

Iterating

```
>>> import numpy as np

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> for element in b.flat:
...     print(element)
...
1
2
3
4
5
6
```

ndarray: main numpy object.

Creation of arrays

```
>>> import numpy as np

>>> c = np.array([ [1,2], [3,4] ], dtype=np.complex)
>>> c
array([[ 1.+0.j,  2.+0.j],
       [ 3.+0.j,  4.+0.j]])
```

dtype can be specified when creating an array

ndarray: main numpy object.

Creation of arrays

```
>>> import numpy as np

>>> np.zeros( (3,4) )
array([[0.,  0.,  0.,  0.],
       [0.,  0.,  0.,  0.],
       [0.,  0.,  0.,  0.]])
>>> np.ones( (2,3,4), dtype=int16 )
array([[[ 1, 1, 1, 1],
        [ 1, 1, 1, 1],
        [ 1, 1, 1, 1]],
       [[ 1, 1, 1, 1],
        [ 1, 1, 1, 1],
        [ 1, 1, 1, 1]]], dtype=int16)
>>> np.empty( (2,3) )
array([[  3.73603959e-262,   6.02658058e-154,   6.55490914e-260],
       [  5.30498948e-313,   3.14673309e-307,   1.00000000e+000]])
```

create a 0 matrix of 3x4 dimentions

ndarray: main numpy object.

Creation of arrays

```
>>> import numpy as np

>>> np.arange(0,10,0.1)
array([ 0. ,  0.1,  0.2,  0.3,  0.4,  0.5,  0.6,  0.7,  0.8,  0.9,  1. ,
        1.1,  1.2,  1.3,  1.4,  1.5,  1.6,  1.7,  1.8,  1.9,  2. ,  2.1,
        2.2,  2.3,  2.4,  2.5,  2.6,  2.7,  2.8,  2.9,  3. ,  3.1,  3.2,
        3.3,  3.4,  3.5,  3.6,  3.7,  3.8,  3.9,  4. ,  4.1,  4.2,  4.3,
        4.4,  4.5,  4.6,  4.7,  4.8,  4.9,  5. ,  5.1,  5.2,  5.3,  5.4,
        5.5,  5.6,  5.7,  5.8,  5.9,  6. ,  6.1,  6.2,  6.3,  6.4,  6.5,
        6.6,  6.7,  6.8,  6.9,  7. ,  7.1,  7.2,  7.3,  7.4,  7.5,  7.6,
        7.7,  7.8,  7.9,  8. ,  8.1,  8.2,  8.3,  8.4,  8.5,  8.6,  8.7,
        8.8,  8.9,  9. ,  9.1,  9.2,  9.3,  9.4,  9.5,  9.6,  9.7,  9.8,
        9.9])
```

get an array, from 0 to 10 (not included) with decimal

ndarray: main numpy object.

Creation of arrays

```
>>> import numpy as np

>>> np.arange(0,10,0.1).reshape(10,10)
array([[ 0. ,  0.1,  0.2,  0.3,  0.4,  0.5,  0.6,  0.7,  0.8,  0.9],
       [ 1. ,  1.1,  1.2,  1.3,  1.4,  1.5,  1.6,  1.7,  1.8,  1.9],
       [ 2. ,  2.1,  2.2,  2.3,  2.4,  2.5,  2.6,  2.7,  2.8,  2.9],
       [ 3. ,  3.1,  3.2,  3.3,  3.4,  3.5,  3.6,  3.7,  3.8,  3.9],
       [ 4. ,  4.1,  4.2,  4.3,  4.4,  4.5,  4.6,  4.7,  4.8,  4.9],
       [ 5. ,  5.1,  5.2,  5.3,  5.4,  5.5,  5.6,  5.7,  5.8,  5.9],
       [ 6. ,  6.1,  6.2,  6.3,  6.4,  6.5,  6.6,  6.7,  6.8,  6.9],
       [ 7. ,  7.1,  7.2,  7.3,  7.4,  7.5,  7.6,  7.7,  7.8,  7.9],
       [ 8. ,  8.1,  8.2,  8.3,  8.4,  8.5,  8.6,  8.7,  8.8,  8.9],
       [ 9. ,  9.1,  9.2,  9.3,  9.4,  9.5,  9.6,  9.7,  9.8,  9.9]])
```

`ndarray`: main numpy object.

Methods:

- **Array Creation**: `arange, array, copy, empty, empty_like, eye, fromfile, fromfunction, identity, linspace, logspace, mgrid, ogrid, ones, ones_like, r , zeros, zeros_like`

- **Conversions**: `astype, atleast_1d, atleast_2d, atleast_3d, mat`

- **Manipulations**: `array_split, column_stack, concatenate, diagonal, dsplit, dstack, hsplit, hstack, item, newaxis, ravel, repeat, reshape, resize, squeeze, swapaxes, take, transpose, vsplit, vstack`

- **Questions:** `all, any, nonzero, where`

`ndarray`: main numpy object.
Methods:

- **Ordering:** `argmax, argmin, argsort, max, min, ptp, searchsorted, sort`
- **Operations:** `choose, compress, cumprod, cumsum, inner, fill, imag, prod, put, putmask, real, sum`
- **Basic Statistics:** `cov, mean, std, var`
- **Basic Linear Algebra:** `cross, dot, outer, svd, vdot`

ndarray: main numpy object.

Examples: **reshape**

```
>>> import numpy as np

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )
>>> b
array([[1, 2],
       [3, 4],
       [5, 6]])

>>> b.reshape(2,3)
array([[1, 2, 3],
       [4, 5, 6]])

>>> b.reshape(2,4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: total size of new array must be unchanged
```

ndarray: main numpy object.

Examples: **reshape**

```
>>> import numpy as np

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )
>>> b
array([[1, 2],
       [3, 4],
       [5, 6]])

>>> b.reshape(2,3)
array([[1, 2, 3],
       [4, 5, 6]])

>>> b.reshape(2,-1)
array([[1, 2, 3],
       [4, 5, 6]])
```

ndarray: main numpy object.

Examples: Arithmetic operations

```
>>> import numpy as np

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> b+1
array([[2, 3],
       [4, 5],
       [6, 7]])

>>> b*2
array([[ 2,  4],
       [ 6,  8],
       [10, 12]])

>>> b*b
array([[ 1,  4],
       [ 9, 16],
       [25, 36]])
```

ndarray: main numpy object.

Examples: matrix operations

matrix multiplication (2,3) x (3,2) -> (2,2)

```
>>> import numpy as np

>>> a = np.array([1, 2, 3, 4])

>>> a.dot(a)
30
```

ndarray: main numpy object.

Examples: matrix operations

matrix multiplication (2,3) x (3,2) -> (2,2)

```
>>> import numpy as np

>>> b = np.array( [ [1,2], [3,4], [5,6] ] )

>>> b.dot(b)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: objects are not aligned

>>> c = b.transpose()
>>> c
array([[1, 3, 5],
       [2, 4, 6]])

>>> b.dot(c)
array([[ 5, 11, 17],
       [11, 25, 39],
       [17, 39, 61]])
```

`ndarray`: main numpy object.
More examples:

https://numpy.org/doc/

Why use numpy for working with multidimensional arrays ?

- **Higher performance**: Built on optimized C and Fortran libraries. Faster operations on large arrays. **Vectorization:** operations on entire arrays at once without using explicit Python loops

- **Efficient memory usage**: Compact memory layout more memory-efficient than Python lists or nested lists

pandas

https://pandas.pydata.org/

https://pandas.pydata.org/docs/user_guide/index.html

`pandas`

What **`pandas`** offers.

Packager for data manipulation and analysis.

- **Data structures** and **operations** for manipulating numerical tables and time series

What **pandas** offers.

DataFrames: Two-dimensional, size-mutable, potentially heterogeneous tabular data.

- Summay: **Describe**
- Indexing
- Plotting
- Aggregation: **GroupBy**
- ...

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html

pandas

What **pandas** offers.

DataFrames: Two-dimensional, size-mutable, potentially heterogeneous tabular data.

```
import pandas as pd
df = pd.DataFrame({"col1": [1, 2], "col2": [3, 4]})
df
```

	col1	col2
0	1	3
1	2	4

Example: The iris dataset

```
import sklearn
from sklearn import datasets
df = sklearn.datasets.load_iris(as_frame=True)["data"]
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

Example: The iris dataset

```
df.describe()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
df.loc[0:2, ['sepal length (cm)', 'sepal width (cm)']]
```

	sepal length (cm)	sepal width (cm)
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2

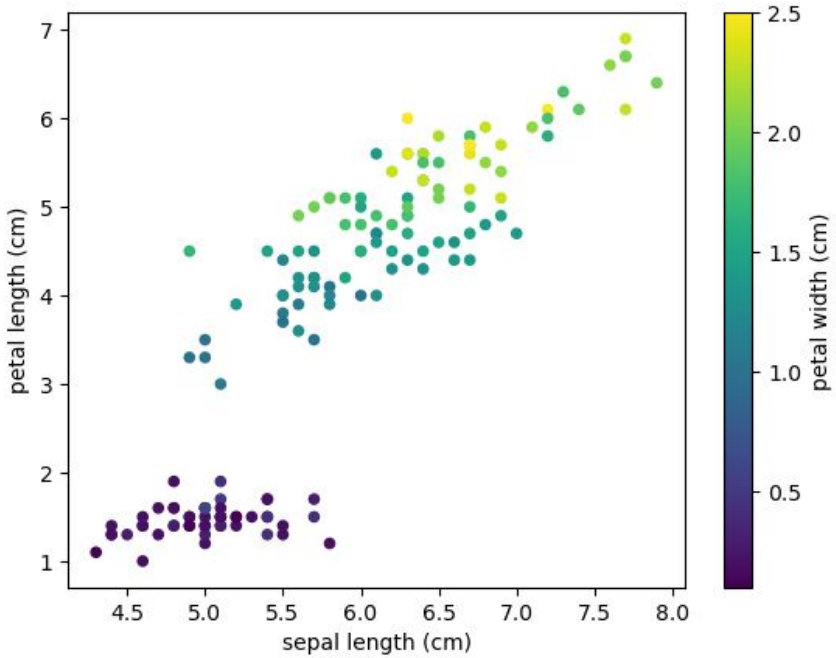
first 3 rows and this two specific columns

df.loc[:, ["sepal length (cm)"]]

all the rows and this special column

Example: The iris dataset

```
df.plot.scatter(x='sepal length (cm)', y='petal length (cm)', c='petal
width (cm)', colormap='viridis')
```



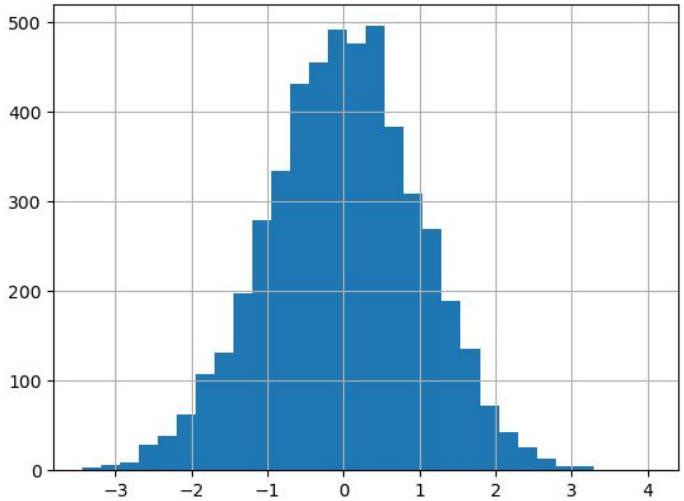
matplotlib

https://matplotlib.org

https://matplotlib.org/stable/api/index.html

matplotlib


```
from numpy.random import randn
from matplotlib import pyplot as plt

x = randn(5000)

plt.hist(x, bins=30)
plt.grid()
plt.show()
```



https://matplotlib.org/stable/gallery/index.html

SciPy

http://www.scipy.org/

https://docs.scipy.org/doc/scipy/reference/index.html

`scipy`

What **`scipy`** offers.

Scipy core packages include:

- numpy
- SciPy library: library for scientific computing

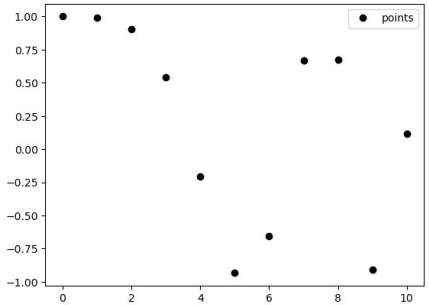
scipy

What **scipy** offers.

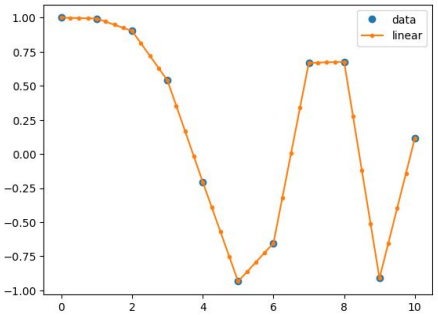
- **scipy.constants:** physical and mathematical constants and units
  http://docs.scipy.org/doc/scipy/reference/constants.html
- **scipy.special**: the definition of numerous special functions of mathematical physics
- **scipy.integrate**: several integration techniques including an ordinary differential equation integrator
- **scipy.optimize**: optimization algorithms.
- **scipy.interpolate:** interpolation facilities

Example: scipy.interpolate

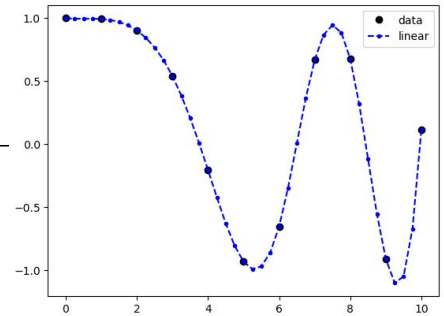
```
x = np.linspace(0, 10, num=11, endpoint=True)
y = np.cos(-x**2/9.0)
f = interp1d(x, y)
f2 = interp1d(x, y, kind="cubic")

plt.plot(x,y, "o", color="black")
plt.legend(["points"], loc="best")
```




```
xnew = np.linspace(0, 10, num=41, endpoint=True)
plt.plot(x,y, "o")
plt.plot(xnew, f(xnew), ".-",)
plt.legend(["data", "linear"], loc="best")
```




```
xnew = np.linspace(0, 10, num=41, endpoint=True)
plt.plot(x,y, "o")
plt.plot(xnew, f2(xnew), ".--",)
plt.legend(["data", "cubic"], loc="best")
```



`scipy`

What **`scipy`** offers.

- **`scipy.fftpack:`** Fourier transform
- **`scipy.signal:`** Signal processing
- **`scipy.linalg:`** Linear algebra
- **`scipy.stats:`** statistics
- **`scipy.spatial:`** triangulations, voronnoi diagrams, convex hulls, KDTree,...
- **`scipy.spatial.distance:`** distance computations
- etc.

scipy

Example: **scipy.linalg**

$$\begin{aligned} x + 3y + 5z &= 10 \\ 2x + 5y + z &= 8 \\ 2x + 3y + 8z &= 3 \end{aligned}$$

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 5 & 1 \\ 2 & 3 & 8 \end{bmatrix}^{-1} \begin{bmatrix} 10 \\ 8 \\ 3 \end{bmatrix} = \frac{1}{25} \begin{bmatrix} -232 \\ 129 \\ 19 \end{bmatrix} = \begin{bmatrix} -9.28 \\ 5.16 \\ 0.76 \end{bmatrix}.$$

```
import numpy as np

from scipy import linalg

A = np.array( [ [1,3,5], [2,5,1], [2,3,8] ] )
B = np.array( [10, 8, 3] )


#Using matrix inverse
solution = linalg.inv(A).dot(B)

solution2 = linalg.solve(A,B)
```

Machine Learning in Python
https://scikit-learn.org/stable/index.html

An machine learning platform
https://www.tensorflow.org/

An open source machine learning framework
https://pytorch.org/